Project 1: Processor Scheduling, Performance, and Energy Trade-offs

Complete six C++ decisions, verify the simulator, and use the results to choose a processor design.

1. Scenario and requirements

An industrial IoT processing module receives a batch of 16 prepared sensor samples every 1,000 ns. Compare In-order, Scoreboarding, and Tomasulo-style scheduling. Choose the correct design that meets the response deadline and uses the least energy over one task period.

Requirement	Project setting
Primary workload	16 serial-feedback samples; 6 instructions per sample; 96 instructions total
Clock	4 GHz for all three models; one cycle is 0.25 ns
Deadline	140 ns for the primary comparison
Task period	1,000 ns between batches; batches do not overlap
Sleep state	0.10 mW after completion until the period ends; zero transition time and energy
Decision objective	Minimum period energy among correct, deadline-feasible candidates

The deadline is the latest permitted completion time. The task period is the interval between batches, not another deadline. Study B also asks you to make a choice under a 70 ns deadline.

These are educational workloads, timing rules, and energy assumptions. They are not measured requirements or power values for a commercial IoT chip. Branches, stores, cache misses, speculation, ROB, and exception recovery are not a consideration for this project.

Start with the unedited code

```
make demo
```

Run commands from the Project1_ACA folder containing Makefile. The starter runs all three models but contains incorrect or overly restrictive decisions. Its results are marked UNVERIFIED. Save screenshot S1 and preserve results/demo/ before editing; do not overwrite this before-repair evidence.

A successful run or Functional oracle: PASS does not establish correct timing or energy. You must repair the decisions and run the validation checks before using final results.

Software: C++17 compiler, make, and Python 3.9+ on Linux/WSL or macOS. Python runs checks and organizes results; the simulator and student edits are C++. No third-party Python packages are required. Due dates and permitted assistance are specified in Blackboard.

AI tools may be used to clarify general course concepts. They are not to be used to generate or modify your submitted code, answer the project questions, or write your analysis.

2. Workloads and processor resources

The serial workload is workloads/iot_serial.asm. Its inputs are 2, 4, 6, ..., 32, and its initial state is 4.0. Each sample updates that state:

```
state_next = ((state + sample) * 0.5 + 1.0) / 2.0
```

One sample uses the six instructions below. The file explicitly repeats them for addresses 0 through 15, giving 16 x 6 = 96 instructions. There is no simulated loop or branch.

```
LD    F8, [k]          # read the sample
FADD  F9, F0, F8       # previous state + sample
FMUL  F10, F9, F24     # multiply by 0.5
FADD  F11, F10, F25    # add 1.0
FDIV  F0, F11, F26     # divide by 2.0; update state
IADD  R1, R1, R2       # update the final sample count
```

Here [k] stands for an actual word index [0] through [15]. F24=0.5, F25=1.0, F26=2.0, R1=0, and R2=1. The .reg and .mem directives initialize values before time zero; they are not executed instructions. The final state is in F0 and the final count is in R1. The oracle checks all architectural registers.

The previous sample's FDIV produces F0, which the next sample's first FADD reads. This is the cross-sample RAW dependency. R1 is a final count, not an intermediate completion signal. The task completes only after every instruction has published its result.

Operation	Execution latency	Energy per operation
IADD	1 cycle	2 pJ
FADD / FSUB	3 cycles	8 pJ
FMUL	5 cycles	20 pJ
FDIV	12 cycles	60 pJ
LD	8 cycles	40 pJ

Each model has one INT, FP ADD/SUB, FP MUL, FP DIV, and MEMORY unit. Different units can overlap. Each unit is non-pipelined and stays occupied through result publication. At most one instruction issues, one launches, and one publishes per cycle. The In-order model is a multicycle execution-side model, not a literal five-stage MIPS pipeline; early ALU bypass is not modeled.

The four-stream workload

workloads/iot_four_streams.asm uses four feedback states, F0-F3, with four samples per state. It still contains 16 samples and 96 instructions. It changes the application dependencies and results; it is not an equivalent optimization of the serial recurrence. The streams share execution resources and temporary register names. Neither workload uses multiple C++ threads or simulated cores.

Keep both workload files, their instruction order, and the baseline coefficients unchanged.

3. Coding tasks

Replace only the marked return expressions in the three student files. The six functions inspect arguments and return true or false. Equivalent correct expressions are accepted. G1-G6 are coding-task labels, not ABET outcome numbers.

Task	File under src/	Decision to complete
G1	inorder_control.cpp	May the already-selected head instruction launch?
G2	scoreboard_control.cpp	May the next instruction issue?
G3	scoreboard_control.cpp	Does an older source operand block a completed writer?
G4	tomasulo_control.cpp	May the next instruction enter a reservation station?
G5	tomasulo_control.cpp	Should a source wait for its producer or capture a value now?
G6	tomasulo_control.cpp	May a broadcast update the destination register?

src/controllers.cpp already selects candidates, resolves competing requests, binds sources before destination renaming, and updates tables and waiting operands. The engine supplies the clock, arithmetic, memory, FU timing, event counts, and output. Do not add scheduling loops, timing counters, or energy charges.

Parameters used by the six decisions

Parameter	Meaning in this simulator
sources_ready	The selected instruction's source operands are available.
fu_free / row_free	The required FU is available; row_free also includes its Scoreboard row.
older_inflight / any_inflight	Older work / any other admitted instruction has not yet published its result.
destination_tag	The pending writer ID for this instruction's destination register; 0 means none.
ready_unread	An older source value is available but has not yet been read or captured.
same_register	That older source register has the same name as the completed writer's destination.
rs_free	A reservation-station entry is available.
producer_tag	The pending producer ID for a source register; 0 means none.
current_tag / result_tag	The destination's current RAT entry / the ID of the producer now broadcasting.

Repair order

Complete G1, then G2-G3. For Tomasulo, repair G5-G6 before G4: the starter's admission restriction can hide value/tag errors. Partial repairs may cause wrong results or explicit errors.

```
make check MODEL=inorder
make check MODEL=scoreboard
make decisions-test MODEL=tomasulo
```

Use docs/Guided_Implementation.md and docs/Model_Contract.md for the full interfaces and timing contract. Protected source files, configurations, tests, and expected results must remain unchanged.

4. State, timing, and energy

Register values, tags, and waiting instructions

The Register File stores current numeric values. The Register Alias Table (RAT) records the latest pending producer for each architectural register. A Reservation Station (RS) holds an instruction and its source values or waiting tags. RAT and RS are different structures.

A tag is a producer identifier, not a value, address, or count. Tag 0 means no pending producer. Tag 3 means the producer identified by 3, not three producers. For example, Register File[F6] = 10.0 and RAT[F6] = 3 mean that 10.0 is still stored while a newer F6 value is pending from producer 3.

In Scoreboarding, the destination tag corresponds to the Register Result Status entry. ready_unread describes one older source: it is false while that source is waiting, true when its value is available but unread, and false again after capture. A missing operand does not protect an old value.

In Tomasulo, each waiting consumer keeps its own producer tag. The register's current RAT entry may later point to a different writer. Broadcasting to waiting consumers and updating the architectural destination are separate actions. The provided code always delivers the broadcast to matching consumers; G6 controls only the destination update.

Timing

Issue admits an instruction into a waiting structure. Launch starts execution on a FU. Completion ends the fixed execution latency. Publication makes the result available. These are separate events.

A newly issued instruction cannot launch in the same cycle. An operation launched at cycle c with latency L completes at the end of c+L-1 and can publish no earlier than c+L. A consumer awakened by publication can launch no earlier than the next cycle. A freed slot or FU cannot be reused in the publication cycle. Source binding sees a result published in its issue cycle before the new destination mapping is installed.

Energy calculation

```
T_ns = cycles / frequency_GHz
E_dynamic_pJ = sum(event_count * energy_per_event_pJ)
E_background_pJ = background_mW * T_ns
E_task_pJ = E_dynamic_pJ + E_background_pJ
P_task_avg_mW = E_task_pJ / T_ns
E_period_pJ = E_task_pJ + sleep_mW * (period_ns - T_ns)
```

1 mW x 1 ns = 1 pJ; 1,000 pJ = 1 nJ. Do not multiply accumulated energy by time again. Sleep power is 0.10 mW. Period energy is undefined if execution exceeds the period.

The observer counts modeled hardware events, not C++ loop iterations or host execution time. Tag comparisons charge the full modeled bit width, including nonmatches; they do not count letters in tag names. For the baseline control_scale = 1, background power is 20 mW (In-order), 24 mW (Scoreboard), and 30 mW (Tomasulo with 8 RS entries). All coefficients are teaching assumptions.

5. Validation and results

After completing the coding tasks, run the following commands from the Project1_ACA folder containing Makefile. Rerun both commands after any later code change. Include the validation outcome in your report.

```
make check
make evaluate
```

make check runs 40 decision cases and 39 short-program comparisons, plus framework tests. It checks instruction timing, final registers, hardware-event counts, and energy. make evaluate reruns validation and writes results/evaluation/comparison.csv and per-run files.

config/baseline.cfg sets control_scale = 1.0. Leave it unchanged. The v2.0 script creates 12 rows: use only the six control_scale = 1 rows for Studies A and B. The extra 0.5 and 2 rows do not require analysis. Each generated per-run .cfg records that run's settings.

Status	Meaning
Functional oracle: PASS	Final registers match the sequential interpreter; timing and energy are not certified.
CHECK: PASS	The supplied decision, short-program, and framework checks passed.
VERIFIED_FOR_THIS_RUN	The build checks and this run's reference timing, event, and numeric checks passed.
UNVERIFIED / RUN_ERROR	Debugging results only, not a validated final baseline.

Passing finite tests is not a proof for every possible input. Standalone runs and make demo remain UNVERIFIED because they do not perform full acceptance. Report unresolved failures honestly; do not label their measurements as validated.

Files to inspect locally

File	Use
comparison.csv / *_summary.csv	Cycles, time, energy, power, and deadline feasibility.
*_energy.csv	Event counts, per-event energy, and component totals for Study A.
*_timing.csv	One row per instruction: issue, start, complete, and write cycles.
*_trace.csv	Raw cycle-level state; optional debugging, not required reading for Q2.
*_registers.csv	Final architectural register values.

Use these files to prepare your report. Raw CSV, JSON, and log files do not need to be submitted. Include the relevant numbers, tables, and supporting screenshots in the report instead.

For a readable timing table, use tools/show_run.py. Q2 gives a test file, instruction index, and viewing command for each choice. This viewer displays saved summary, timing, and energy data; it does not run the simulation or decode the raw trace.

6. Experiments and architecture selection (45 points)

Use your verified results to complete Studies A-C. Report units and compare architectures using the same workload and configuration. Choose feasibility first, then minimum period energy. Support conclusions with your data; identify untested changes as hypotheses. Include the relevant tables and explanations in your report; the raw run files are for your own analysis, not a required submission.

For Studies A and B, use control_scale = 1 (this is a default scale, so you may not need to change it for the experiments). deadline_met refers to 140 ns; use deadline_70_met or compare time_ns with 70 for the tighter requirement. If no candidate meets the deadline, state that no baseline design is feasible.

Study A. Serial workload (20 points)

Use your verified iot_serial results at control_scale = 1.

a. Report cycles, execution time, task energy, period energy, and whether each architecture meets 140 ns. (5 points)

b. Which architecture do you recommend? Explain why it is preferable to the other two under the stated requirements. (7 points)

c. Compare the energy consumption of two architectures using at least two components from their _energy.csv files (e.g., scheduler probes, tag comparisons, or register accesses). Calculate the energy of one component using its event count and energy per event. Explain how these components contribute to the energy difference between the architectures. (8 points)

Study B. Four independent streams (15 points)

Use your verified iot_four_streams results at control_scale = 1.

a. Report execution time, period energy, and whether each architecture meets the 140 ns and 70 ns deadlines. (5 points)

b. With a 140 ns deadline, which architecture would you choose to minimize period energy? Explain. (5 points)

c. With a 70 ns deadline, which architecture would you choose? Identify the feasible candidates and justify your choice. (5 points)

Study C. Architectural recommendation (10 points)

Under what conditions would the architectural recommendation change?

7. Short-answer questions (20 points)

Answer Q1-Q5. Use brief explanations, calculations, or labeled diagrams. Typed text and legible handwriting are accepted. Include the specific values or states needed to support your reasoning.

Q1. Workload and requirements (4 points)

Consider workloads/iot_serial.asm and workloads/iot_four_streams.asm.

a. Which register carries the state from one sample to the next, and what data dependency does this create? (1 point)

b. At 4 GHz, how many clock cycles are allowed by a 140 ns deadline? Show your calculation. (2 points)

c. How do the data dependencies in the four-stream workload differ from those in the serial workload? (1 point)

Q2. A scheduling decision (4 points)

Choose ONE task below. Read its specified test program and use your own timing results. The table identifies where to look; explain the scheduling behavior yourself.

Task	Test program	Instructions / columns to examine
G1 In-order	tests/cases/overlap.asm	Compare I2 (IADD R1, R2, R3) Start with I1 (LD F0, [0]) Done.
G3 Scoreboard	tests/cases/war.asm	Compare I3 (FADD F5, F6, F7) Done / Write with I2 (FDIV F4, F1, F5) Start.
G4 Tomasulo	tests/cases/load_admission.asm	Compare I3 (LD F1, [1]) Issue / Start with I1 (LD F0, [0]) Write.

a. Describe the relevant operand and resource conditions for the selected instruction. (1 point)

b. Explain why the instruction can proceed or must wait at the relevant cycle: launch for G1, result publication for G3, or issue into an RS for G4. (2 points)

c. Identify the test file, instruction number, and cycle or cycles supporting your explanation. Include your timing-table screenshot; it may also serve as S4. (1 point)

Viewing results for Q2

After make check passes, run only the command for your chosen task from the Project1_ACA / folder:

```
# G1 - In-order
python3 tools/show_run.py results/check/overlap_inorder

# G3 - Scoreboard
python3 tools/show_run.py results/check/war_scoreboard

# G4 - Tomasulo
python3 tools/show_run.py results/check/load_admission_tomasulo
```

I = instruction number; Issue = admission; Start = execution begins; Done = execution finishes; Write = result publication. Instruction numbers start at 1 and exclude .reg, .mem, and comments; they are not source-file line numbers.

Pass the output prefix, not a name ending in _timing.csv or _trace.csv. show_run.py displays saved summary, timing, and energy data; it does not rerun or validate the simulation, or decode the raw trace. Read the test instructions with the timing table. Reading the complete raw trace is not required.

7. Short-answer questions (continued)

Q3. Producer tags and register versions (4 points)

An instruction needs a source value whose pending producer tag is 3. Later, producer 3 broadcasts its result, but its destination register's current RAT entry is tag 7.

a. Before the broadcast, should the consumer read the register-file value or wait? (1 point)

b. What happens to consumers waiting for tag 3 when that result is broadcast? (1 point)

c. Should this result update the destination register? (2 points)

Q4. Time, power, and energy (4 points)

Assume 400 cycles at 4 GHz, 3,000 pJ of dynamic energy, and 20 mW of background power. The task period is 1,000 ns, and the sleep power (energy-use rate) is 0.10 mW.

a. Calculate execution time, task energy, average execution power, and period energy. Show your calculations and units. (3 points)

b. Why is adding every instruction's execution time/latency not a correct way to calculate elapsed time when operations overlap? (1 point)

Q5. Validation and the before/after comparison (4 points)

Choose one architecture. Compare your unedited demo with your final verified serial run at control_scale = 1.

a. Report the before/after cycles and period energy. (2 points)

b. Identify one repaired decision that helps explain the change. (2 points)

For Q5(a), you may refer to an existing before/after table in your report rather than repeat the same data.

8. Submission and grading

Submit the following four files: the three edited C++ source files listed below and one report PDF. Update the project code in your assigned GitHub repository. Submit one ZIP on Blackboard containing only the three edited C++ files and Project1_Report.pdf.

```
inorder_control.cpp
scoreboard_control.cpp
tomasulo_control.cpp
Project1_Report.pdf
```

The report must contain Q1-Q5, Studies A-C, and supporting screenshots or figures. Use the headings in this description. About 3-5 pages of analysis plus tables and screenshots is a guide, not a strict limit. Typed explanations, labeled drawings, and legible handwriting are accepted.

Follow this description if the README or other files list different requirements.

Run evidence within the report

Capture the following evidence from your own runs. Commands generate output, not screenshots; use your operating system's screenshot tool. Place the images in the report with brief captions. One image may support more than one answer.

Evidence	When and command	What to show
S1. Before repair	Before editing: make demo	Three-model table with UNVERIFIED visible. Keep this initial output for Q5; do not overwrite it with a repaired run.
S2. Final validation	After repairs: make check	Decision and short-program test totals and the final CHECK status for the submitted code.
S3. Final evaluation	After validation: make evaluate	The six serial/four-stream rows at control_scale = 1, including units and validation status. Two screenshots are acceptable.
S4. Scheduling evidence	Use the show_run.py command for your Q2 choice.	The selected instruction timing table, test/model, and relevant cycles. Reuse the Q2 screenshot; raw trace decoding is not required.

Keep command or file context readable. Do not change data values or conceal relevant warnings. Screenshots are part of report quality.

Grading

Assessment component	Points
Coding tasks: G1-G4 = 3 each; G5-G6 = 4 each	20
Short-answer questions: Q1-Q5 = 4 each	20
Experiments and architecture selection: A 20; B 15; C 10	45
Submission and Report quality: project folder updated in GitHub repo; zip submission in Blackboard; supporting evidence; organization and clarity; readability	15
Total	100